

Medii de proiectare și programare

2025-2026

Curs 12

Conținut

- Websocket (cont.)
- Autentificare REST
 - JWT
 - Spring Security +JWT

Java WebSockets API - Anotări

- Un endpoint care acceptă cereri WebSocket (serverul) poate fi orice POJO (Plain Old Java Object) annotat cu `@ServerEndpoint`.
- Această anotare specifică containerului (web) că clasa annotată ar trebui considerată un endpoint WebSocket.
- Elementul anotării specifică calea către endpoint-ul WebSocket.

```
@ServerEndpoint("/hello")
```

```
public class MyEndpoint { }
```

```
@ServerEndpoint("/hello/{userid}")
```

```
public class MyEndpoint { }
```

- valoarea variabilei `{userid}` poate fi obținută în metodele corespunzătoare ciclului de viață folosind anotarea `@PathParam`.

Java WebSockets API - Adnotări

- Un endpoint care inițiază o conexiune WebSocket (clientul) poate fi un POJO adnotat cu `@ClientEndpoint`.
- `ClientEndpoint` nu accepta variabile de cale (ex. {userid}) deoarece nu așteaptă cereri.

`@ClientEndpoint`

```
public class MyClientEndpoint {}
```

- Inițierea unei conexiuni WebSocket, folosind abordarea bazată pe adnotări:

```
javax.websocket.WebSocketContainer container =  
javax.websocket.ContainerProvider.getWebSocketContainer();
```

```
container.connectToServer(MyClientEndpoint.class,  
new URI("ws://localhost:8080/hello"));
```

- Clasele adnotate cu `@ServerEndpoint` sau `@ClientEndpoint` sunt numite endpoint-uri adnotate.

Java WebSockets API - Adnotări

- După ce a fost stabilită o conexiune WebSocket se creează un obiect de tip `Session` și metoda adnotată cu `@OnOpen` va fi apelată.
- Metoda poate avea diferiți parametri:
 - Un parametru de tip `javax.websocket.Session` specificând sesiunea (obiectul Session) creată.
 - Un obiect de tip `EndpointConfig` care conține informații despre configurația endpoint-ului.
 - Zero sau mai mulți parametri de tip String adnotați cu `@PathParam`, care referă variabilele de cale din calea endpoint-ului(server).

`@OnOpen`

```
public void myOnOpen (Session session) {  
    System.out.println ("WebSocket opened: "+session.getId());  
}
```

Java WebSockets API - Adnotări

- O instanță de tip **Session** este validă atâta timp cât conexiunea WebSocket nu este închisă.
- Clasa **Session** conține metode care permit dezvoltatorilor să obțină mai multe informații despre conexiune.
- Conține metoda **getUserProperties()** care returnează un dicționar **Map<String, Object>** ce păstrează date specifice aplicației.
- Permite dezvoltatorilor să păstreze informații specifice în obiectul de tip **Session**, informații care ar trebui păstrate între diferite apeluri ale metodei.

Java WebSockets API - Adnotări

- Când unul dintre **participanți primește un mesaj**, este apelată metoda adnotată cu **@OnMessage**.
- Metoda poate avea **parametrii**:
 - Un parametru de tip **javax.websocket.Session**.
 - Zero sau mai mulți parametri de tip string adnotați cu **@PathParam**, care referă valorile variabilelor de cale (server).
 - Mesajul care poate fi de tip text, binar sau pong.
- Pentru fiecare tip de mesaj este permisă câte o metodă adnotată cu **@OnMessage**. Parametrii permiși pentru specificarea conținutului depind de tipul mesajului primit.

@OnMessage

```
public void myOnMessage (String txt) {  
    System.out.println ("WebSocket received message: "+txt);  
}
```


Java WebSockets API - Anotări

- Dacă metoda anotată cu `@OnMessage` returnează ceva, valoarea returnată va fi transmisă celuilalt participant la conexiune.

`@OnMessage`

```
public String myOnMessage (String txt) {  
    return txt.toUpperCase();  
}
```

- Metodă alternativă de a trimite mesaje folosind o conexiune WebSocket:

```
RemoteEndpoint.Basic other = session.getBasicRemote();  
other.sendText ("Hello, world");
```

- Metoda `getBasicRemote()` din clasa `Session` returnează o reprezentare a celuilalt participant la conexiunea WebSocket, un `RemoteEndpoint`. Acea instanță `RemoteEndpoint` poate fi folosită pentru trimiterea diferitor tipuri de mesaje (text, binar, pong).
- Obiectul de tip `Session` poate fi obținut/păstrat în metodele corespunzătoare ciclului de viață. (ex. metoda anotată cu `@OnOpen`)

Java WebSockets API - Adnotări

- Metoda `@OnClose` este apelată când se închide conexiunea WebSocket.
- Poate avea parametrii:
 - Un obiect de tip `javax.websocket.Session`. Acest parametru nu mai poate fi folosit după ce conexiunea WebSocket este închisă (imediat după terminarea execuției metodei).
 - Un obiect de tip `javax.websocket.CloseReason` care descrie motivul închiderii conexiunii WebSocket (ex. închidere normală, eroare de protocol, serviciu supraîncărcat, etc.).
 - Zero sau mai mulți parametri de tip String adnotați cu `@PathParam`, referind variabilele de cale din URL asociat endpoint-ului (server).

`@OnClose`

```
public void myOnClose (CloseReason reason) {  
    System.out.println ("Closing a WebSocket due to  
    "+reason.getReasonPhrase());  
}
```

- În cazul apariției unei erori, va fi apelată metoda adnotată cu `@OnError`.

Java WebSockets API - Mesaje

- Orice obiect Java poate fi transmis sau primit ca și mesaj WebSocket.
- Trei tipuri diferite de mesaje:
 - *Text*
 - *Binare*
 - *Pong* (specifice conexiunii WebSocket).
- Pentru mesaje de tip text sunt permisi următorii parametrii:
 - **String** care reprezintă tot mesajul
 - Un tip primitiv Java sau clasa asociată care reprezintă mesajul convertit la acel tip
 - **String** și o valoare booleană care reprezintă o parte din mesaj (mesajul poate avea mai multe părți)
 - **Reader** pentru a primi întregul mesaj ca și un stream (blochează execuția până la terminarea citirii de pe stream)
 - orice obiect Java pentru care participantul la conexiune are un *text decoder* (**Decoder.Text** sau **Decoder.TextStream**).

Java WebSockets API - Mesaje

- Parametrii permisi pentru **mesajele binare**:
 - **byte[]** sau **ByteBuffer** pentru a primi tot mesajul
 - perechea (**byte[]**, **boolean**) sau (**ByteBuffer**, **boolean**) pentru a primi mesajul în părți.
 - **InputStream** pentru a citi tot mesajul dintr-un stream (*blocking*).
 - orice obiect pentru care participantul are un *binary decoder* (**Decoder.Binary** sau **Decoder.BinaryStream**).
- Parametrii permisi pentru **mesaje pong**:
 - **PongMessage** pentru tratarea mesajelor de tip pong.
- Orice obiect Java poate fi convertit într-un mesaj de tip text sau binar folosind un convertor (eng. *encoder*).
- Mesajul în format text/binar va fi transmis celuilalt participant, unde va fi reconvertit (eng. *decoded*) într-un obiect Java.
- Pentru transmiterea mesajelor WebSocket se folosește adesea formatul XML sau JSON. În acest caz convertirea/reconvertirea se reduce la transformarea unui obiect Java într-un XML/JSON și invers.

Java WebSockets API - Mesaje

- Un convertor este o clasă care implementează interfața `javax.websocket.Encoder`, iar un reconvertor este o clasă care implementează interfața `javax.websocket.Decoder`.
- Participanții la conexiunea WebSocket trebuie să știe posibili convertori/reconvertori.
- Lista convertorilor/reconvertorilor poate fi transmisă ca și elemente ale adnotărilor `@ClientEndpoint` și `@ServerEndpoint`.

```
@ServerEndpoint(value="/endpoint", encoders = MessageEncoder.class,  
decoders= MessageDecoder.class)  
public class MyEndpoint {  
    ...  
}
```

Java WebSockets API - Messages

```
class MessageEncoder implements Encoder.Text<MyJavaObject> {  
    @Override  
    public String encode(MyJavaObject obj) throws EncodingException {  
        ...  
    }  
}
```

```
class MessageDecoder implements Decoder.Text<MyJavaObject> {  
    @Override  
    public MyJavaObject decode (String src) throws DecodeException {  
        ...  
    }  
  
    @Override  
    public boolean willDecode (String src) {  
        // return true if we want to decode this String into  
        // a MyJavaObject instance  
    }  
}
```

Java WebSockets API - Messages

- Interfață **Encoder** are următoarele subinterfețe:
 - **Encoder.Text** - convertirea obiectelor Java în mesaje de tip text
 - **Encoder.TextStream** - adăugarea obiectelor Java la un *character stream*
 - **Encoder.Binary** - convertirea obiectelor Java în mesaje binare
 - **Encoder.BinaryStream** - adăugarea obiectelor Java la un *binary stream*
- Interfață **Decoder** are 4 subinterfețe:
 - **Decoder.Text** - convertirea mesajelor text într-un obiect Java
 - **Decoder.TextStream** - citirea unui obiect Java dintr-un *character stream*
 - **Decoder.Binary** - convertirea unei mesaj binar într-un obiect Java
 - **Decoder.BinaryStream** - citirea unui obiect Java dintr-un *binary stream*

Abordarea bazată pe interfețe

- Clasa `javax.websocket.Endpoint`:

- se redefinesc metodele `onOpen`, `onClose`, și `onError`:

```
public class MyOwnEndpoint extends javax.websocket.Endpoint {  
    public void onOpen(Session session, EndpointConfig config) {...}  
    public void onClose(Session session, CloseReason closeReason) {...}  
    public void onError (Session session, Throwable throwable) {...}  
}
```

- Pentru interceptarea mesajelor trebuie înregistrat un obiect de tip `javax.websocket.MessageHandler` în definirea metodei `onOpen`:

```
public void onOpen (Session session, EndpointConfig config) {  
    session.addMessageHandler (new MessageHandler() {...});  
}
```


Abordarea bazată pe interfețe

- Interfața **MessageHandler** are două subinterfețe:
 - **MessageHandler.Partial** - apariția unui mesaj parțial.
 - **MessageHandler.Whole** - apariția unui mesaj complet.

```
public void onOpen (Session session, EndpointConfig config) {  
    final RemoteEndpoint.Basic remote = session.getBasicRemote();  
    session.addMessageHandler (new MessageHandler.Whole<String>() {  
        public void onMessage(String text) {  
            try {  
                remote.sendString(text.toUpperCase());  
            } catch (IOException ioe) {  
                // handle send failure here  
            }  
        }  
    });  
}
```


Spring WebSocket API

- Interfața **WebSocketHandler** pentru procesarea mesajelor și gestiunea evenimentelor din ciclul de viață
 - **TextWebSocketHandler** - procesarea mesajelor text
 - **BinaryWebSocketHandler** - procesarea mesajelor binare
- Metode:
 - **afterConnectionEstablished(WebSocketSession session)**
 - **handleMessage(WebSocketSession session, WebSocketMessage<?> message)**
 - **supportsPartialMessages()**
 - **afterConnectionClosed(WebSocketSession session, CloseStatus closeStatus)**
 - **handleBinaryMessage(WebSocketSession session, BinaryMessage message)**
 - **handleTextMessage(WebSocketSession session, TextMessage message)**
 - **handlePongMessage(WebSocketSession session, PongMessage message)**

Spring WebSocket API

```
public class ChatWebsocketHandler extends TextWebSocketHandler {
    private static List<WebSocketSession> sessions = new CopyOnWriteArrayList<>();
    @Override
    public void handleTextMessage(WebSocketSession session, TextMessage message) {
        System.out.println("[Websocket] New TextMessage: " + message.getPayload());
    }
    @Override
    public void afterConnectionEstablished(WebSocketSession session) {
        sessions.add(session);
    }
    @Override
    public void afterConnectionClosed(WebSocketSession session, CloseStatus status) {
        sessions.remove(session);
    }
    public void sendUsersAll(User[] users) {
        ObjectMapper objectMapper = new ObjectMapper();
        try{
            String result=objectMapper.writeValueAsString(users);
            for(WebSocketSession websocketSession : sessions) {
                try {
                    websocketSession.sendMessage(new TextMessage(result));
                }catch (IOException e){
                    System.err.println("error sending message to websocket"+e);
                }
            }
        } catch(JsonProcessingException e){
            System.err.println("error writing object"+e);
        }
    }
}
```

Spring WebSocket API

- Interfața **WebSocketConfigurer** pentru configurarea WebSocket

@Configuration

@EnableWebSocket

```
public class WebSocketConfiguration implements WebSocketConfigurer {
```

```
    @Bean
```

```
    public ChatWebsocketHandler createChatWebsocketHandler() {
```

```
        return new ChatWebsocketHandler();
```

```
    }
```

```
public void registerWebSocketHandlers(WebSocketHandlerRegistry registry) {
```

```
    registry.addHandler(createChatWebsocketHandler(), "/chatws");
```

```
                                //.setAllowedOrigins("*");
```

```
}
```

```
}
```

WebSockets

- Exemple:

- Broadcast
- Whiteboard
- Chat users

- Referințe

- RFC 6455: <https://tools.ietf.org/html/rfc6455>
- Johan Vos, *JSR 356, Java API for WebSocket*, <http://www.oracle.com/technetwork/articles/java/jsr356-1937161.html>
- Brian Raymor, *WebSockets: Stable and Ready for Developers*, <https://msdn.microsoft.com/en-us/aa969243.aspx>
- Spring WebSockets: <https://docs.spring.io/spring-boot/reference/messaging/websockets.html>

STOMP + WebSocket

- STOMP (Simple Text Oriented Messaging Protocol) - protocol bazat pe frame-uri pentru trimiterea mesajelor

SEND

destination: /queue/trade

content-type: application/json

content-length: 44

{"action": "BUY", "ticker": "MMM", "shares", 44}^@

- Se pot folosi brokeri de mesaje (ActiveMQ, RabbitMQ)
- Interfața **WebSocketMessageBrokerConfigurer** pentru configurare
 - **registerStompEndpoints(...)**
 - **configureMessageBroker(...)**

<https://stomp.github.io/>

<https://docs.spring.io/spring-framework/reference/web/websocket/stomp.html>

Autentificare REST

- *Autentificarea* (eng. *Authentication*): verificarea credențialelor trimise de client (utilizator, parolă)
- *Autorizarea* (eng. *Authorization*): verificarea dreptului de a accesa anumite resurse (deja autentificat)
- Antetul HTTP *Authorization*
- Constrângerea REST *stateless*: la fiecare cerere HTTP trebuie transmise informațiile necesare
- Abordări:
 - HTTP Authentication (RFC 2617)
 - Basic Authentication
 - Digest Authentication
 - OAuth 2.0

Autentificare REST

- **HTTP Basic Authentication**

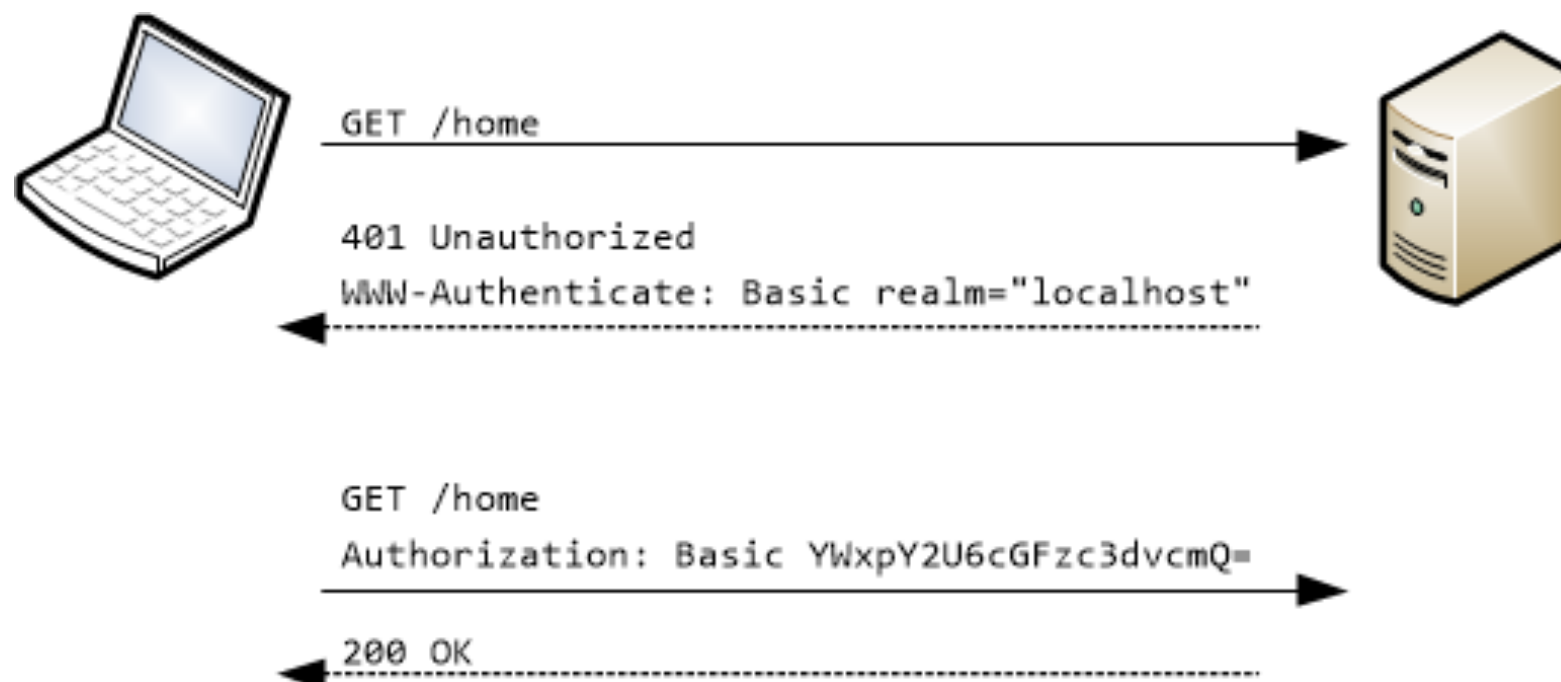
- Folosirea antetului HTTP Authorization în care se transmit username-ul și parola codificate în base64 la fiecare cerere:

GET / HTTP/1.1

Host: example.org

Authorization: Basic Zm9vOmJhcG==

- Credențialele sunt codificate, dar **nu sunt criptate!**
- Informațiile pot fi obținute ușor.
- Se recomandă folosirea doar cu SSL/TLS



Autentificare REST

- **HTTP Digest Authentication**

- Folosirea antetului HTTP Authorization în care se transmit username-ul, parola și alte informații codificate.
- HMAC (hash based message authentication)
- Se transmite o versiune criptată a parolei
- Se codifică și alte informații despre resursa dorită

Exemplu: utilizator “ion”, parola “ionpass”,

GET /users/ion/account

```
digest = base64encode(hmac("sha256", "ionpass",  
                             "GET+/users/ion/account"))
```

GET /users/ion/account HTTP/1.1

Host: example.org

Authorization: hmac ion:[digest]

- Parola nu poate fi criptată pe server, pentru a putea reconstrui digest
- Se recomandă folosirea unui “secret” cunoscut de client și server

Autentificare REST

- **HTTP Digest Authentication**

- **Avantaj:** un hacker nu poate modifica cererea (ar trebui să modifice valoarea *digest* și nu cunoaște “secret”-ul)
- **Dezavantaj:** un hacker poate executa cererea de oricâte ori
 - **Soluție:** se transmit mai multe informații în *digest*:
 - Data curentă
 - **Nonce** (*N*umber used *once*). La cereri subsecvente valoarea nonce este diferită.

```
digest = base64encode(hmac("sha256", "secret",  
    "GET+/users/ion/account+30may201912:59:24+123456"))
```

```
GET /users/ion/account HTTP/1.1
```

```
Host: example.org
```

```
Authorization: hmac ion:123456:[digest]
```

```
Date: 30 may 2019 12:59:24
```

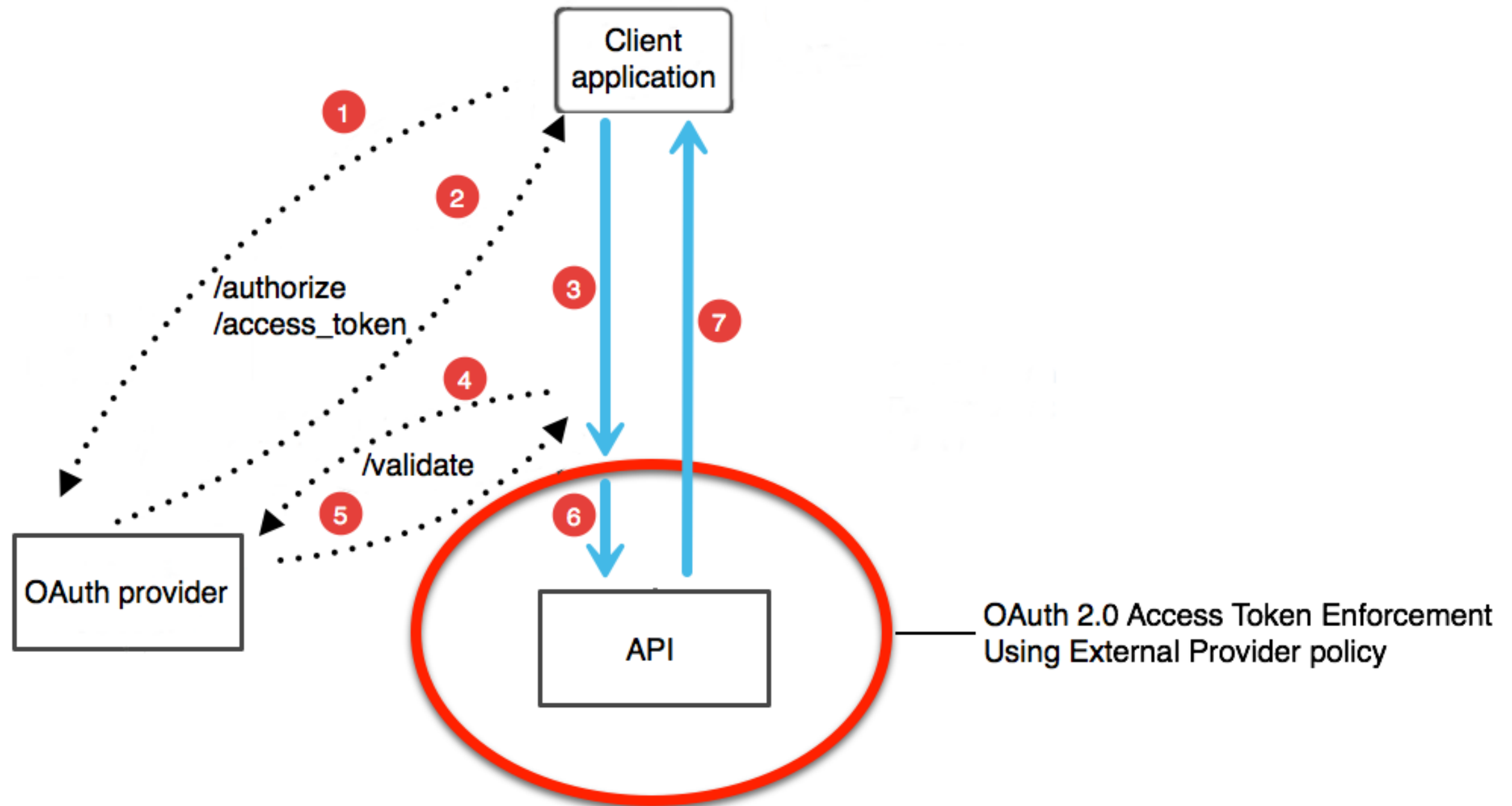
Autentificare REST

- **OAuth**

- OAuth 1.0 decembrie 2007
 - Sigur, dar dificil de implementat (criptografie, interoperabilitate)
- OAuth 2.0 octombrie 2012
 - Mai ușor de implementat
 - Compromisuri la nivel de securitate
 - Cel mai folosit
- **Flux:**
 - Aplicația client se înregistrează la un **furnizor**
 - Furnizorul transmite clientului un “secret” unic
 - Clientul include “secret”-ul la fiecare cerere
 - Dacă cererea nu este formată corect, are date lipsă sau nu conține secretul corect, ea va fi respinsă.

Autenticare REST - OAuth

OAuth Dance



JSON Web Token (JWT)

- **JSON Web Token** este un standard (RFC 7519) care definește un mod *compact* și *independent* de a transmite informații între participanți ca și un object JSON.
 - *Compact*: are dimensiuni reduse.
 - poate fi transmis prin URL, parametrii POST, antet HTTP
 - este transmis rapid
 - *Independent*: un token JWT conține toată informația necesară despre o entitate pentru a evita interogări multiple ale bazei de date.
 - cine primește tokenul nu trebuie să facă alte cereri la server pentru a-l valida.
- Informația conținută într-un token JWT poate fi verificată și se poate avea încredere în ea pentru că este **semnată digital**.
- JWT poate fi semnat folosind:
 - un *secret* (folosind HMAC)
 - cheie publică/privată folosind RSA, ECDSA, etc

JWT

- Aplicabilitate JWT
 - Autentificare/Autorizare
 - Schimb de informații între participanți (client/server)
 - Securizat
- **Structura unui token JWT**
 - 3 părți separate prin '.': **aaa**.**bbb**.**ccc**
 - Header (**aaa**)
 - Payload (**bbb**)
 - Semnătura digitală (**ccc**)

Header JWT

- Conține informații despre tipul de token și algoritmi de criptare folosiți pentru conținut.
- Are **două părți**:
 - tipul tokenului (JWT)
 - algoritmul de criptare (ex. HMAC SHA256, RSA)

```
{  
  "alg": "HS256",  
  "typ": "JWT"  
}
```

Payload JWT

- Conține informații despre entitate ce pot fi verificate (identitatea utilizatorului, permisiunile, etc), eng. *claims*
- 7 *claims* rezervate și recomandate (dar nu obligatorii):
 - *iss* (issuer), *sub* (subject) -utilizatorul, *aud* (audience) - pentru cine a fost creat, *exp* (expiration time), *nbf* (not before time), *iat* (issued at time), *jti* (JWT ID) -identificator unic, poate fi folosit o singură dată

```
{  
  "sub": "1234567890",  
  "name": "John Doe",  
  "admin": true  
}
```

- Poate conține și alte informații adiționale (eng. *custom claims*). Poate fi folosit orice nume, care nu este rezervat prin specificație.

Semnătura digitală JWT

- Este folosită pentru a verifica expeditorul și pentru a verifica dacă mesajul a fost modificat pe parcurs.
- Se codifică Base64 header-ul și payload-ul și se criptează folosind algoritmul specificat în header (împreună cu secretul/cheie publică-privată)

HMACSHA256 (

```
base64UrlEncode(header) + "." +  
base64UrlEncode(payload) ,  
secret)
```

- **Înainte de folosirea unui token JWT trebuie verificată semnătura lui.**

JWT.io

ALGORITHM

HS256



Encoded

PASTE A TOKEN HERE

eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6Ikp1hcm1uZXNjdSBWYXNpbGUuLCJpYXQiOiJlMTYyMzkwMjJ9.J3H8MqVValMLWo1ycEOUBNax3glXeURR2XwU5Z9F9Zc

Decoded

EDIT THE PAYLOAD AND SECRET

HEADER: ALGORITHM & TOKEN TYPE

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

PAYLOAD: DATA

```
{
  "sub": "1234567890",
  "name": "Marinescu Vasile",
  "iat": 1516239022
}
```

VERIFY SIGNATURE

```

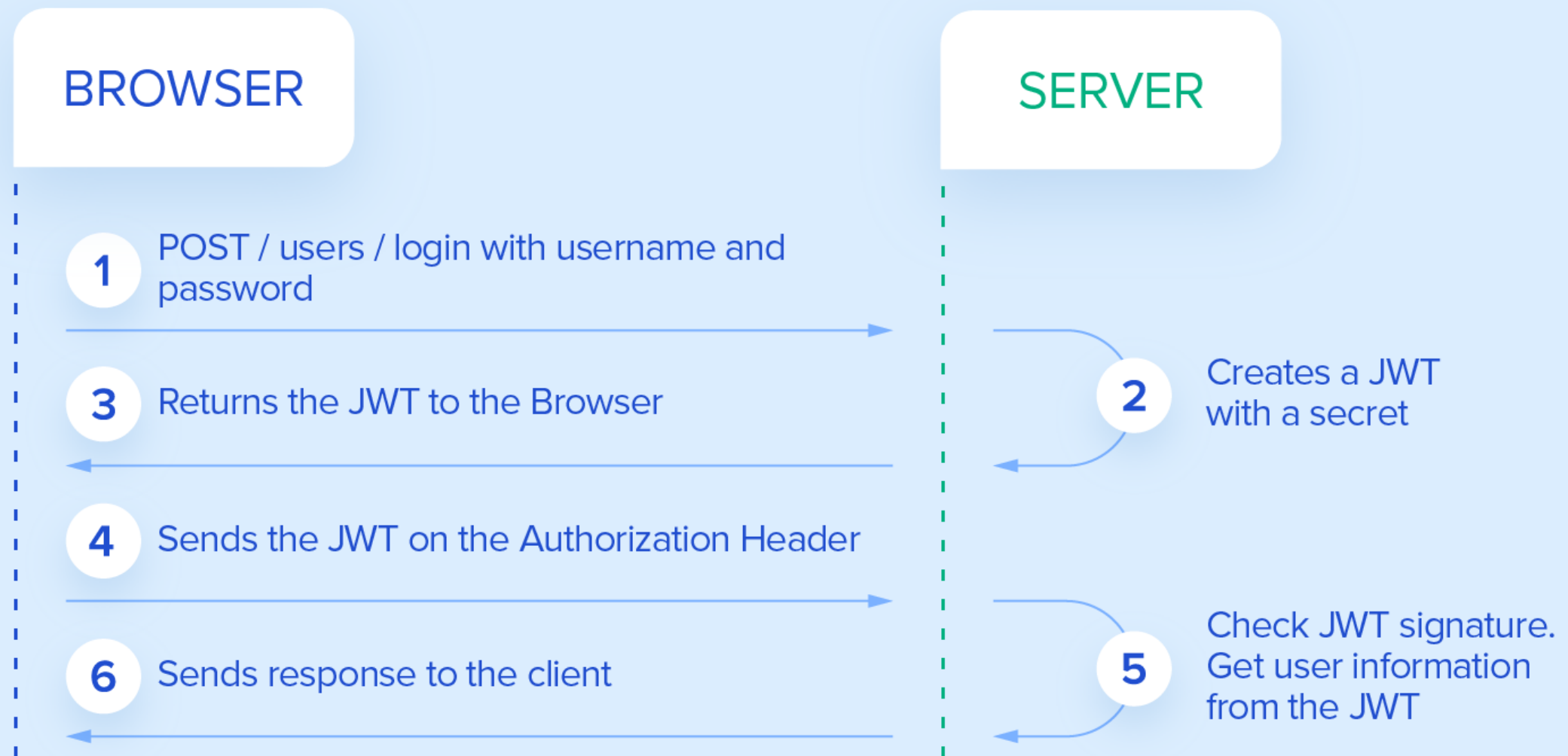
HMACSHA256(
    base64UrlEncode(header) + "." +
    base64UrlEncode(payload),
    secretul meu
) ✓ secret base64 encoded

```

 **Signature Verified**

SHARE JWT

JWT



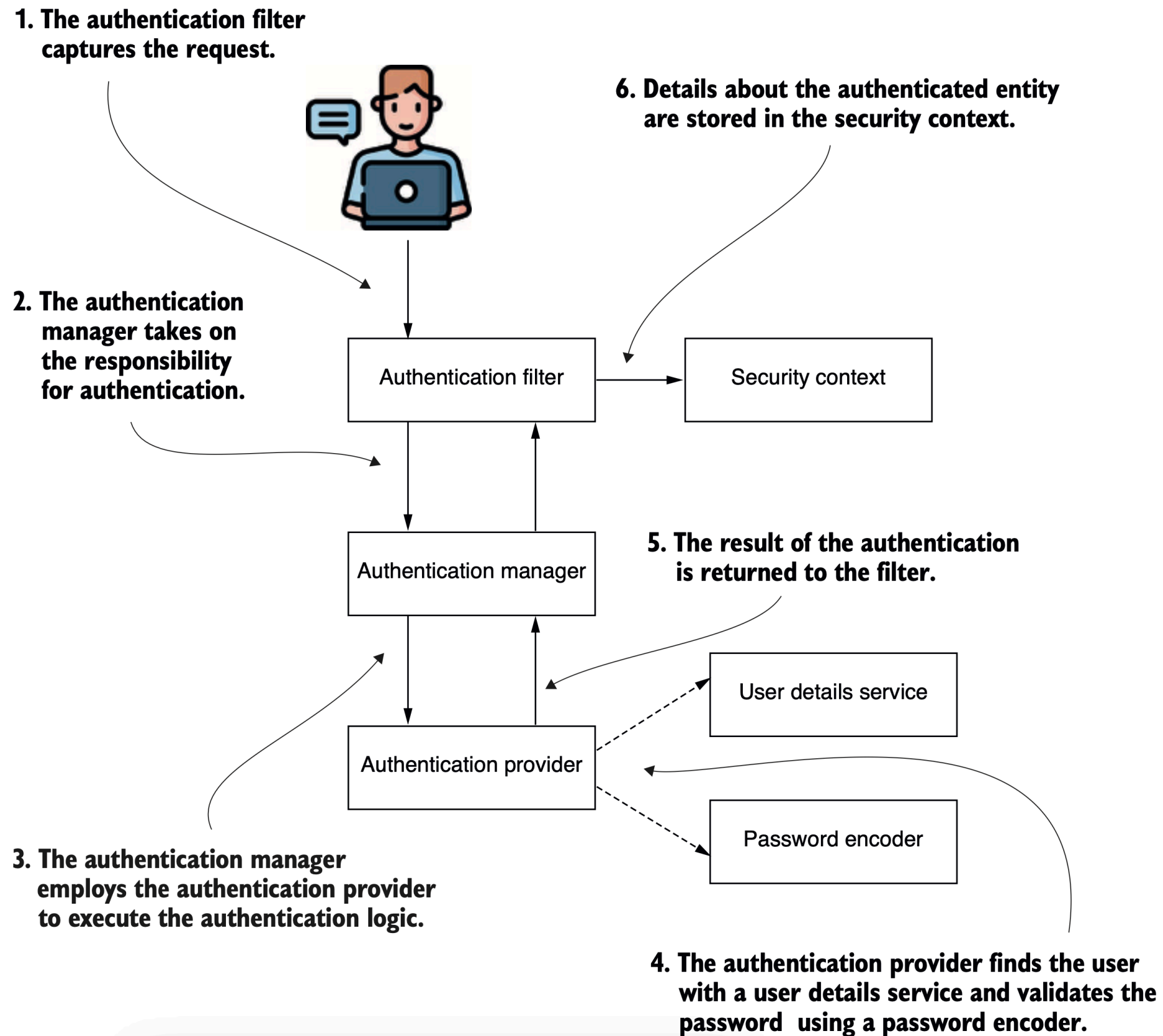
JWT

- Exemplu JWT + Postman

Spring Security

- Framework pentru implementarea securității la nivel de aplicație în Spring
- Oferă o modalitate personalizabilă de implementare a *autentificării, autorizării și protecției împotriva atacurilor comune*
- Este un software open source lansat sub licența Apache 2.0
- <https://spring.io/projects/spring-security>

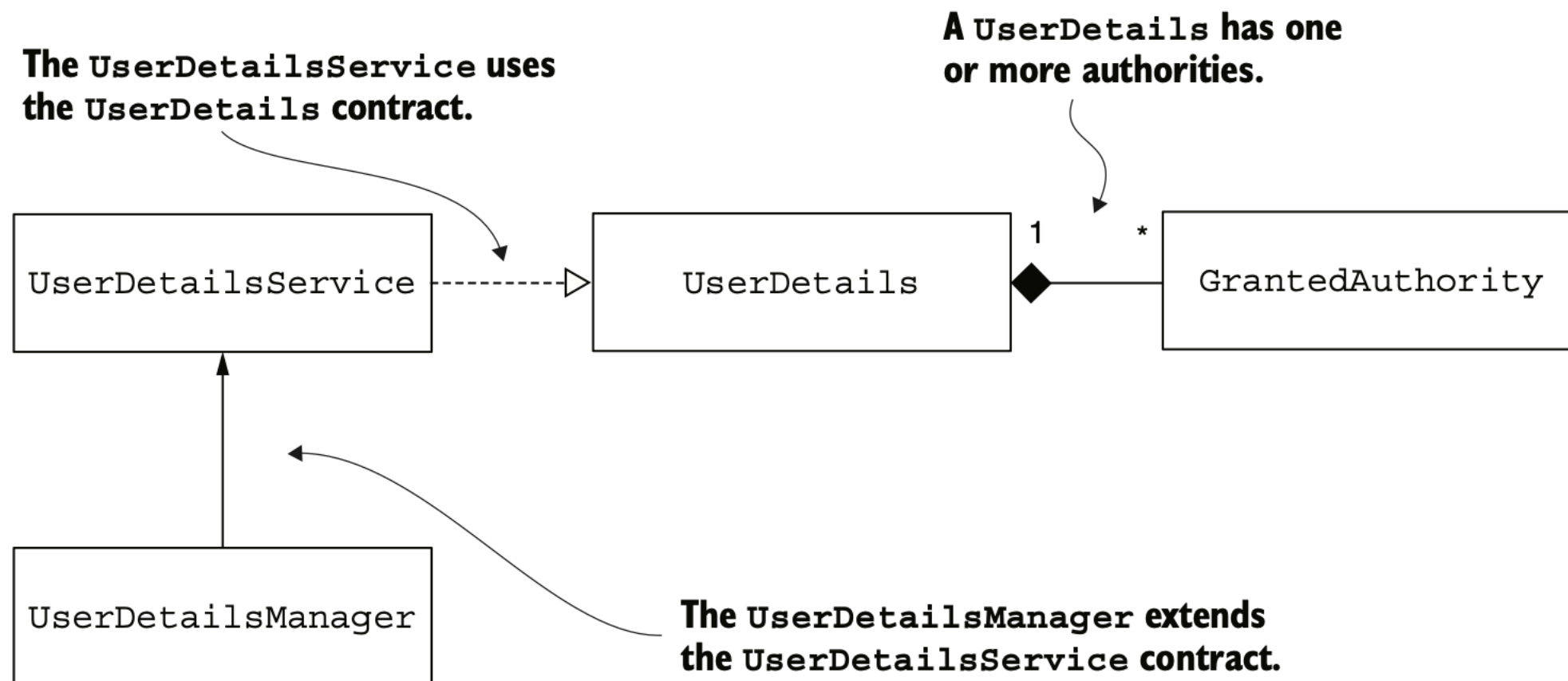
Spring Security -arhitectura generală



Spring Security

1. *Authentication filter* deleagă cererea de autentificare către *AuthenticationManager* și, pe baza răspunsului, configurează contextul de securitate (*Security context*).
2. *AuthenticationManager* folosește *AuthenticationProvider* pentru a procesa autentificarea.
3. *AuthenticationProvider* implementează logica de autentificare.
4. *UserDetailsService* implementează responsabilitatea de gestionare a utilizatorilor (username, parola, drepturi), pe care *AuthenticationProvider* o utilizează în logica de autentificare.
5. *PasswordEncoder* implementează gestionarea parolelor, pe care *AuthenticationProvider* o utilizează în logica de autentificare.
6. *SecurityContext* păstrează datele de autentificare după procesul de autentificare. El va păstra datele până la finalizarea acțiunii (logout, etc).

Spring Security -arhitectura generală



Spring Security

Interfața **UserDetailsService** - încarcă datele specifice unui utilizator

- **loadUserByUsername(String username) : UserDetails**

Interfața **UserDetails** - furnizează informații despre un utilizator

- **getUsername() : String**
- **getPassword() : String**
- **getAuthorities() : Collection<? extends GrantedAuthority>**
- **isEnabled() : boolean**

Clasa **User** poate fi folosită pentru păstrarea informațiilor despre utilizator

(pachetul **org.springframework.security.core.userdetails**)

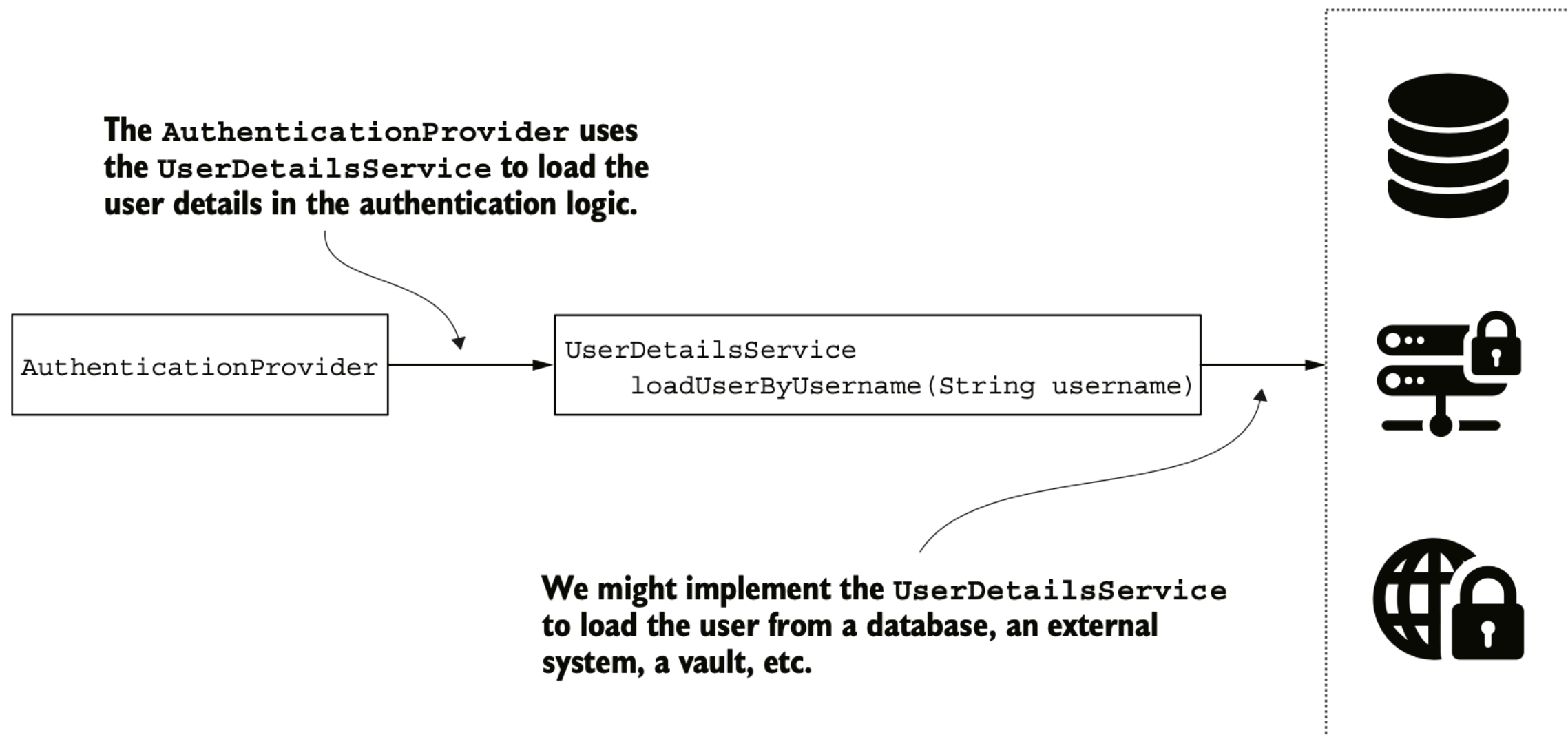
- Folosirea unui DTO sau wrapper, dacă deja există clasa User în aplicație

Interfața **PasswordEncoder** - criptarea parolelor (implementări:

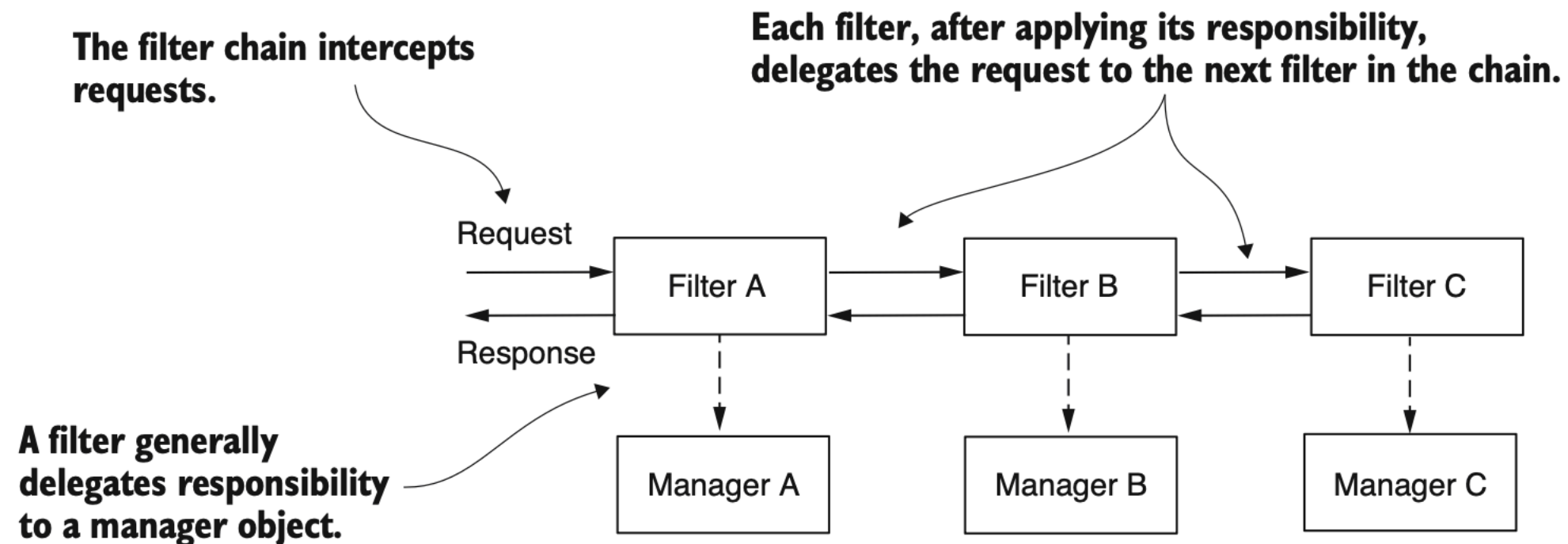
BCryptPasswordEncoder, StandardPasswordEncoder, NoOpPasswordEncoder)

- **encode(rawPWD : CharSequence) : String**
- **matches(rawPWD : CharSequence, encodedPWD : String) : boolean**
- **upgradeEncoding(encodedPWD : String)** //ar trebui criptată din nou

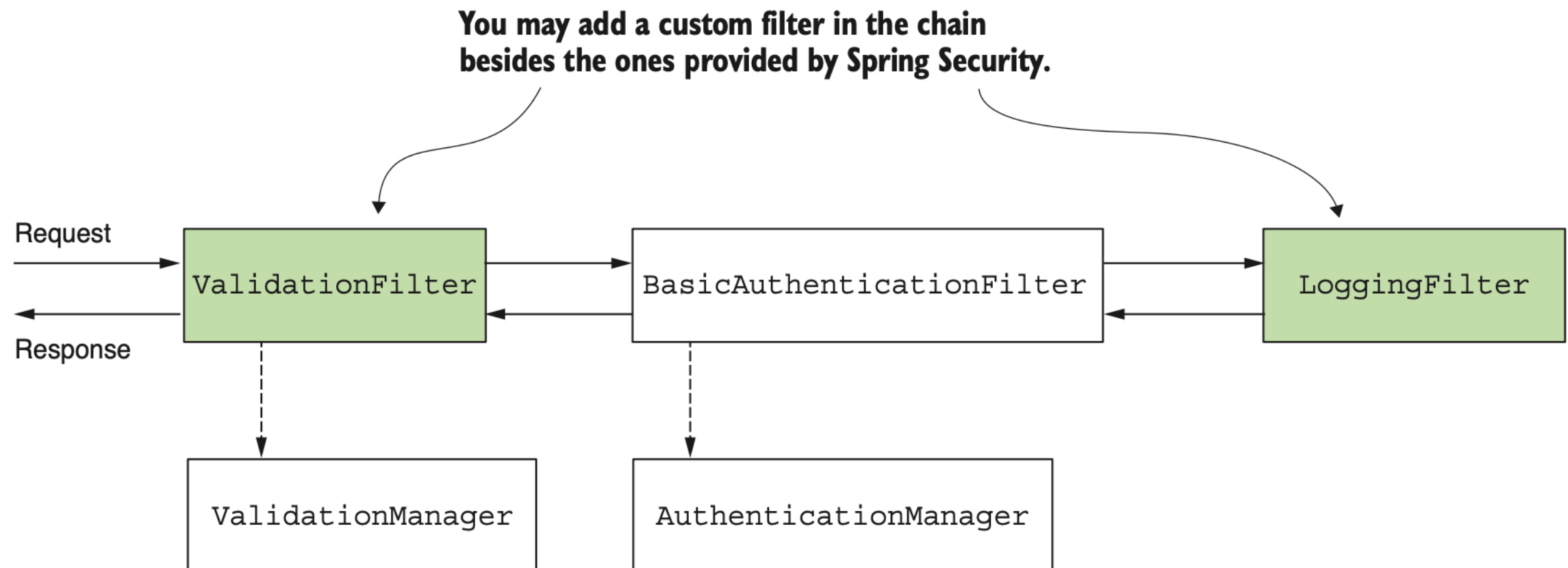
Spring Security -arhitectura generală



Spring Security -Filtre



Spring Security -Filtre



Spring Security

Interfața **AuthenticationManager** - încearcă autentificarea folosind obiectul Authentication transmis (eșec: aruncă excepție)

- **authenticate(auth:Authentication): Authentication**
- Returnează obiectul **Authentication** actualizat cu detaliile utilizatorului
- Va fi păstrat în SecurityContext (Principal)

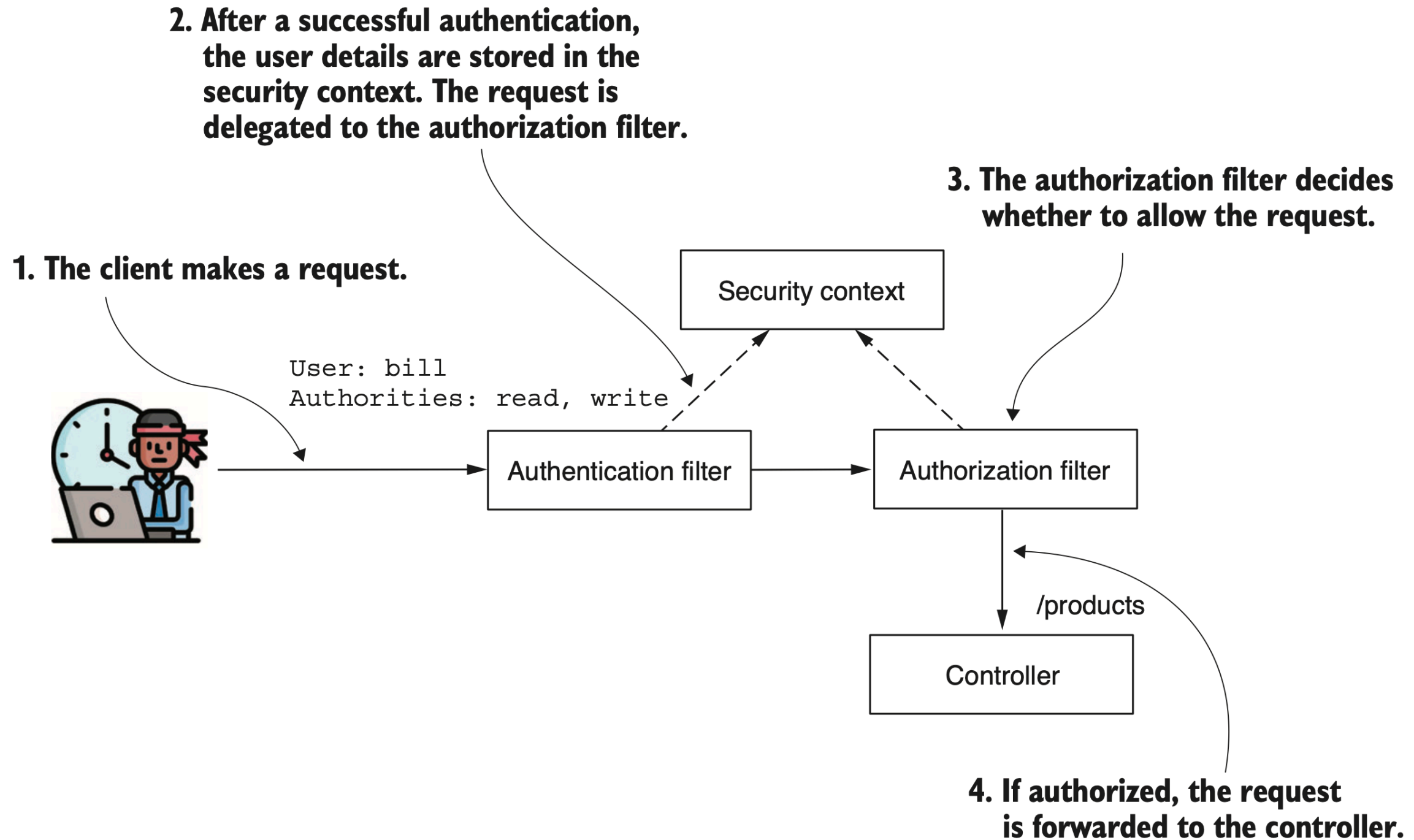
Interfața **Authentication** - reprezintă token-ul pentru o cerere de autentificare, respectiv utilizatorul autentificat (daca autentificarea s-a finalizat cu succes)

- **getDetails()**
- **isAuthenticated()**
- **getCredentials():**
- **getAuthorities():Collection<? extends GrantedAuthority>**
- **etc**

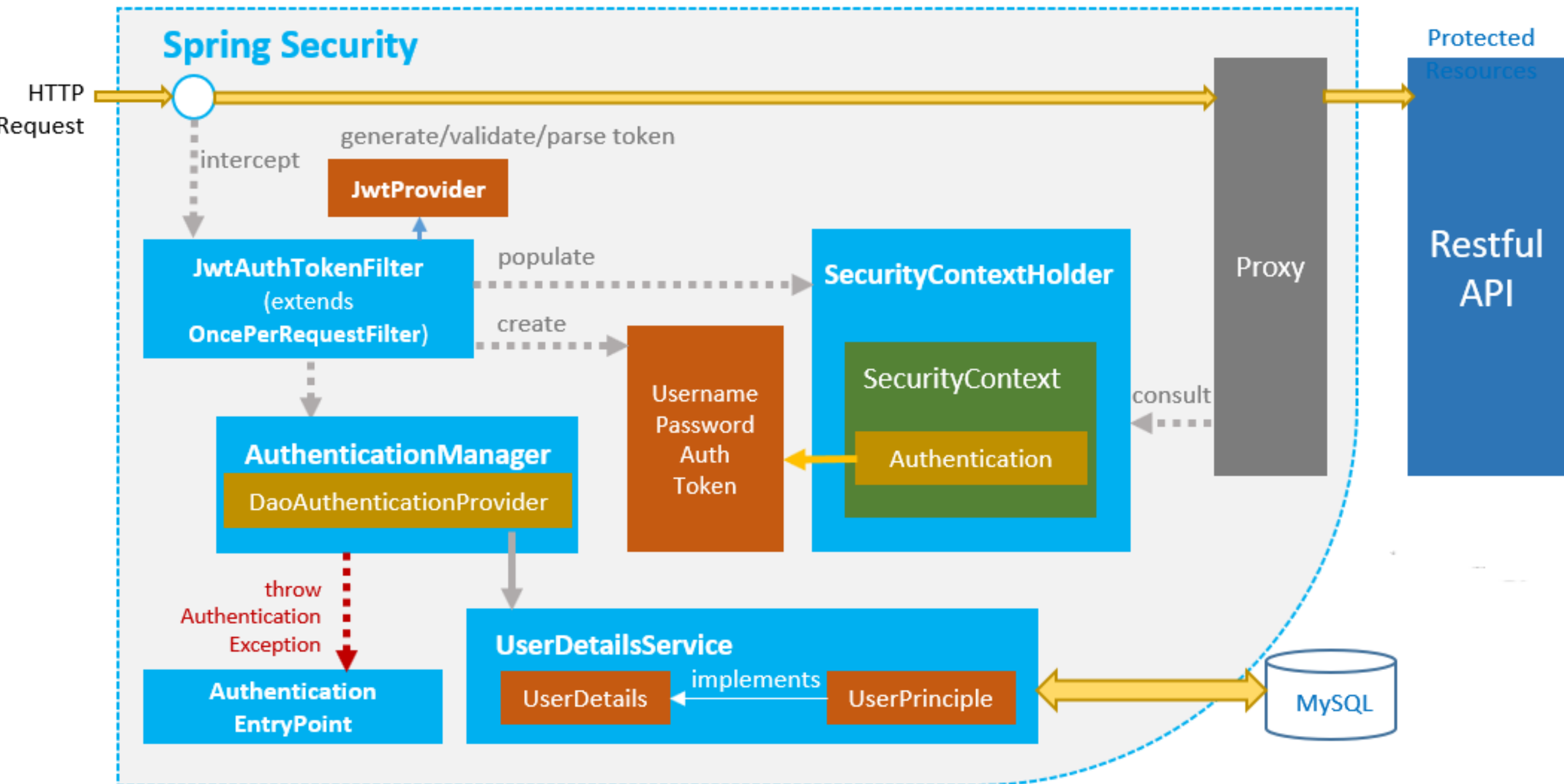
Clase:

- **UsernamePasswordAuthenticationToken**
- **BearerTokenAuthentication**
- **JwtAuthenticationToken**
- **etc**

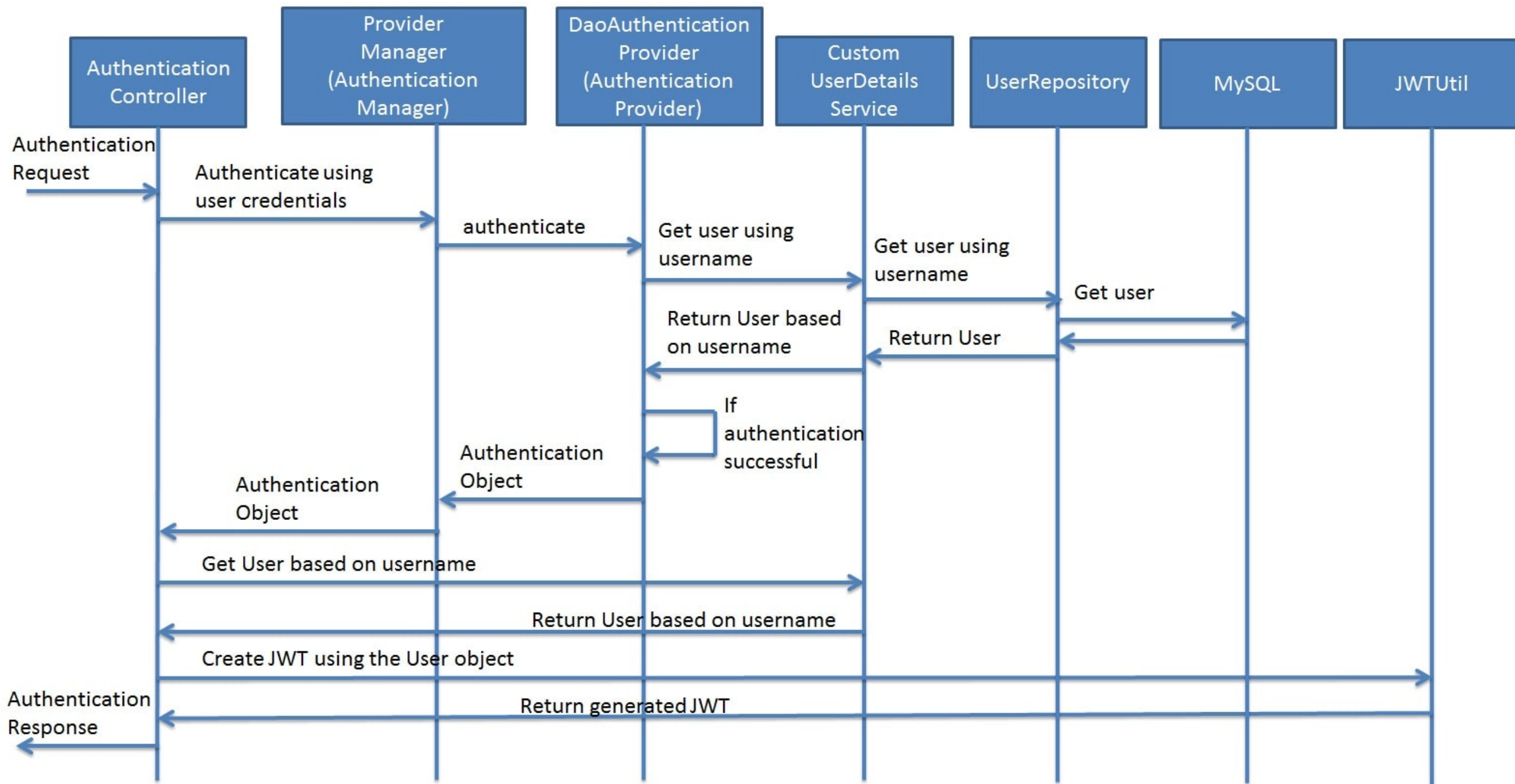
Spring Security -Authorization



Spring Security +JWT



Spring Security +JWT



*<https://www.cloudtechtwitter.com/2023/09/demystifying-jwt-based-authentication.html>

Spring Security +JWT - build.gradle

```
plugins {  
    id 'java'  
    id 'org.springframework.boot' version '3.5.0'  
    id 'io.spring.dependency-management' version '1.1.7'  
}  
group = 'ro.mpp'  
version = '0.0.1-SNAPSHOT'  
repositories {  
    mavenCentral()  
}  
dependencies {  
    implementation 'org.springframework.boot:spring-boot-starter-actuator:3.5.0'  
    implementation 'org.springframework.boot:spring-boot-starter-security'  
    implementation 'org.springframework.boot:spring-boot-starter-web'  
    implementation 'io.jsonwebtoken:jjwt-api:0.12.6'  
    runtimeOnly 'io.jsonwebtoken:jjwt-impl:0.12.6'  
    runtimeOnly 'io.jsonwebtoken:jjwt-jackson:0.12.6'  
    testImplementation 'org.springframework.boot:spring-boot-starter-test'  
    testImplementation 'org.springframework.security:spring-security-test'  
    testRuntimeOnly 'org.junit.platform:junit-platform-launcher'  
}  
tasks.named('test') {  
    useJUnitPlatform()  
}
```


Exemplu Spring Security +JWT

Fisierul application.properties

JWT configuration

jwt.secret=4e075be94ac3f771634aba3dd5b139ba33165e2855d9a95ac5ff028cda491aea

jwt.expiration=3600000

CORS configuration, comma separated if there are more

app.cors.allowed-origins=http://localhost:5173

Spring Security +JWT

Exemplu:

API public:

POST api/login

GET api/messages/test

API necesită autentificare

GET api/messages?to={username}

POST api/messages

Testare folosind client REST (Postman)

Client web

Referințe

- RFC 7235 - Access Authentication Framework
<https://tools.ietf.org/html/rfc7235#section-2>
- RFC 2617 - HTTP Authentication: Basic and Digest Access Authentication
<https://tools.ietf.org/html/rfc2617>
- RFC 6749 - OAuth2 standard
<https://tools.ietf.org/html/rfc6749>
- Laurențiu Spilcă, *Spring Security in Action*, 2nd edition, Manning, 2024
- Guy Levin, *RESTful API Authentication Basics*,
<https://blog.restcase.com/restful-api-authentication-basics/>
- Tutoriale web:
 - <https://www.javaguides.net/2024/05/spring-boot-react-jwt-authentication.html>
 - ...